TECHNICAL REPORT / PILOT ENGINEERING STUDY

Finance Lab MVP Review

Product Findings, Technical Integrity and Release Evidence

Anton Serdyuchenko
ORCID: 0000-0001-7770-2201

Version 1.0 - publication candidate
Report prepared 8 November 2026
Review observations: 7-8 November 2026

Prepared for Zenodo; no DOI has been reserved.

This AI-assisted engineering review records a bounded software snapshot and synthetic checks. It has not undergone peer review. Observed behavior, recommendations and missing evidence are kept distinct. Commercial validation and production release acceptance are not established.

© 2026 Anton Serdyuchenko. The report, its Markdown and LaTeX sources, and document metadata use Creative Commons Attribution 4.0 International (CC BY 4.0). Software and reproduction-code licensing remains separate.

Abstract

This report consolidates the product review and eight technical review steps of Finance Lab's first Budget MVP, recorded on 7-8 November 2026. The reviewed system is a Java/Vaadin modular monolith with a PostgreSQL Anchor Modeling schema, a custom JDBC mapper, a browser-local guest planner and one configured authenticated account. Exact application, Budget and ORM revisions define the runtime baseline; a separate merged documentation revision anchors the review evidence. Recorded normal workflows support precise money handling, explicit saves, historical views and category archiving. Targeted probes reproduced a confirmed-save/read-refresh defect, a rapid dialog-reuse race, an incremental history-transfer limitation and missing initial-category integrity validation. Delivery checks found reproducible accepted binaries alongside Bash 3.2 publication-guard and inherited Maven metadata defects, with persistence integration suites outside the automatic gate. No new reportable security vulnerability was found within the recorded review scope. Local measurements retained 707 latency samples: saved-month reads used 60 prepared statements at 10 categories and 260 at 50, while browser RPC p95 increased from 44.42 to 144.17 milliseconds. Controlled database and explicit JVM restarts preserved selected confirmed state. An isolated encrypted recovery drill preserved data fingerprints for all 24 Budget tables, owners, grants and selected historical projections. Deployed artifact correspondence, constrained-host capacity, provider backup behavior and complete timed operational recovery remain unverified. The report provides a traceable basis for corrective releases and acceptance work; it does not establish market demand, general security, sustainable capacity, independent reproduction or production readiness.

Keywords: personal finance; Budget MVP; modular monolith; Vaadin; PostgreSQL; Anchor Modeling; custom ORM; reproducibility; resilience; software review; synthetic data; AI-assisted engineering.

1. Objectives and scope

Finance Lab supports monthly financial planning: recording income, spending and investments, calculating the remaining balance, explicitly saving a month and inspecting earlier versions. The implemented access model supports one configured account. Guests can work with browser-local data; authenticated storage provides PostgreSQL persistence and cross-browser access for that account. [E0, E2]

The review assesses the implemented planning workflow and its technical foundations. It covers source identity, module boundaries, workflow correctness, persistence integrity, security, delivery, performance and operational recovery. The findings draw on the initial functional review and eight versioned technical reports.

The reviewed runtime is a Java/Vaadin modular monolith with PostgreSQL storage and a custom JDBC mapper. The source revisions and system boundary are defined in Section 3. [E1, E2]

The product assessment examines the implemented workflow and selected usability properties. The evidence does not include interviews, representative-user studies, adoption or retention measurements, price tests or revenue data.

2. Methodology and evidence handling

The review is a descriptive engineering case study of one project, performed with substantial AI assistance. It combines source inspection, existing implementation suites, targeted external probes, public read-only requests and isolated synthetic failure/recovery experiments. Observations oc-

curred on 7-8 November; this synthesis inspected the merged reports on 8 November. Application tests were not rerun merely to write the consolidated report.

Four evidence classes are retained. **Recorded public behavior** describes the initial guest journeys and later anonymous HTTP requests. **Frozen source and completed local checks** describe exact code, artifacts and disposable fixtures. **Prior implementation evidence** includes the candidate verification record in open PR #12; it is not relabeled as a fresh review run. **Unverified operational or product claims** include actual deployed identity, private host/provider settings, demand and human release acceptance.

The technical reports retain original finding IDs, test methods, failed pilot controls, corrected fixtures and limits. Passing a characterization assertion means that the named defective or limited behavior was reproduced. It does not show that the product now meets the intended requirement. Reused suites are identified by their original step; no aggregate passing-test total is constructed by adding reruns of the same cases.

The runtime baseline is distinct from the review-document snapshot. Merged review PRs #13-#20 add reports and external fixtures; merge status is a version-control observation, not evidence of deployment, independent validation or resolution of the findings. Review snapshot D contains the complete eight-step record after PR #20. [E1-E8]

Public artifacts contain synthetic amounts and labels, source revisions and sanitized outcomes. Real passwords, cookies, recovery keys, financial records, private operator paths and provider credentials are excluded. A deployment manifest was unavailable in the reviewed evidence; public page appearance cannot identify server archive bytes.

3. Reviewed system and recorded environments

Snapshot	Repository and exact revision
A: app/runtime	finance-lab-project/app - `f78682778eebed86730ec457f63c96648f19370d`
B: Budget	finance-lab-project/budget - `43d86d28965a241cd45f7938c1adfb2ec50f8132`
O: ORM	finance-lab-project/orm - `8daaf91d07d2e30b563322b22a0c6da7545578e5`
D: merged review	app review evidence - `7edfb504d98ebe88df51a92124e217573addbd41`

Budget's main branch had a different commit ID from B in Step 1, but the two trees were identical. That observation is not a source-content mismatch. The parent POM and both vendored parent copies were also identical. The accepted WAR contains Budget 0.2.0 and ORM 0.1.0; app/authentication and the separately publishable build parent declare 0.1.0. Declared Maven coordinates are not proof that packages or releases were published. [E1, E6]

Area	Recorded configuration and boundary
Application stack	Java 27.0.1.0.3; Jetty 12.1.14 EE11; Servlet API 6.1.0; PostgreSQL 18.6; Maven 3.10.0. The baseline guide names Vaadin 25.3.5; Step 5 records packaged Flow server 25.3.7. These are distinct version labels.
Build/browser tools	Node 24.22.2, npm 11.19.1 and Playwright Java 1.64.0 in the recorded app checks; prior dependency/browser caches were reused.
Persistence/runtime libraries	HikariCP 7.1.0, PostgreSQL JDBC 42.7.14 and Flyway 13.12.0 recorded in the later fixture classpath.
Review host	macOS arm64. Step 7 recorded 12 physical/logical CPU cores and 24 GiB host memory; Docker Linux aarch64 reported 12 CPUs and about 7.75 GiB.

Area	Recorded configuration and boundary
Step 7 limits	App maximum heap 256 MiB; PostgreSQL container memory cap 256 MiB; separate measurement JVM maximum heap 384 MiB. These are not a combined 4 GiB deployment simulation.
Step 8 recovery tools	PostgreSQL client/server 18.6; age 1.2.1; AWS CLI 2.23.6; Bash 5.2.37; Caddy 2.6.2; systemd-analyze 257; ShellCheck 0.10.0; Moto 5.1.14.
Planned production sizing	Two vCPUs at 20% guaranteed performance and 4 GiB RAM for the combined host, proxy, app and database. The local review did not validate that capacity boundary.

Local app checks used disposable localhost TLS; later public requests used default certificate-chain and hostname verification. Exact fixture image IDs and operating-file hashes are in the Step 8 results. Package repositories, warm caches and mutable image tags prevent a claim of hermetic reproduction. [E3-E8]

4. Product and business findings

The initial guest review recorded a synthetic month with income RUB 100,000.50, investments RUB 20,000.00, expenses RUB 45,000.25 and remaining balance RUB 35,000.25. Saved values survived reload; inputs with more than two decimal places were rejected; unchanged saves did not duplicate history. Cancelling an unsaved-change warning retained the draft. Archiving a category preserved its amount in an existing month and removed it from a new month. The planner and login adapted to the narrow-screen check. [E0]

These observations support the tested monthly-planning flow. They do not establish that all navigation sequences work: the later controlled review reproduced the rapid dialog-reuse race. Likewise, the initial absence of a failure in those guest journeys does not establish correctness of authenticated transfer, concurrent saves or production recovery.

The original business review raised four release/product concerns. Its priority labels express local planning judgment, not quantified user impact.

ID / original priority	Recorded concern and bounded implication
B1 / high	Production acceptance was unconfirmed in available release records. Complete and reconcile the existing acceptance evidence; this overlaps S1-01 and S8-01/ S8-02 rather than establishing a separate production defect.
B2 / medium	Guest data remains in a browser profile, and clearing it can remove the only copy. If ongoing public guest use is intended, evaluate export/restore before encouraging reliance on it. This is a conditional product proposal, not a promise to add a feature.
B3 / medium	Budget/ORM integration regression checks depended on manual execution. Step 6 confirmed the automatic-gate gap as S6-03.
B4 / low	The sign-in wording could suggest cross-device access without clearly explaining the single-account access model. Clarifying access expectations is a usability recommendation; no user-comprehension study was performed.

Browser-local storage and the single-account boundary are documented design choices. The concern is the fit between visitor expectations, durability requirements and the intended product audience. The review does not infer that guest storage is encrypted, portable or an account backup.

The product evidence favors a reliability and clarity release around the existing planner before broadening scope. This is an inference from the observed failure modes and evidence gaps, not

proof that a particular roadmap maximizes adoption or revenue. No demand, willingness-to-pay, active-user or production-use outcome is claimed.

5. Architecture and module boundaries

Step 2 inspected imports, qualified references, Maven dependencies, composition, entry points, ports, mapping ownership and transaction callbacks. The production inventory covered 60 Java files: app 13, Budget 18 and ORM 29. Test and runtime-spike sources were excluded from that count; 83 local snapshot files were checked against Git blob hashes. [E2]

The main dependency direction is coherent. App owns assembly, authentication, routing and resource lifecycle. Budget owns financial rules, services, its persistence port and adapters. Its domain types do not import UI, JDBC or ORM code. ORM remains independent of Budget, and Budget's ORM references stay in its PostgreSQL adapter. The app creates and closes the pool and supplies server-derived account identity.

A monthly save, catalog save or initialization enters its own write callback. Current and historical reads use read-only snapshot callbacks and materialize values before returning. Guest transfer orchestrates catalog matching and revision replay in Budget's UI adapter, with separate transactions per replayed revision and final snapshot. It is not an atomic transaction spanning all months.

Two boundary findings remain. S2-01 concerns the shared planner reducing a confirmed Saved/Unchanged result to saved=true, then depending on another read to complete the UI flow; Step 3 confirms its consequence as S3-01. S2-02 is a low-priority dependency declaration problem: app directly uses ORM/Hikari classes while obtaining them transitively through Budget. Neither finding requires replacing the modular monolith or custom ORM.

6. End-to-end workflow correctness

Step 3 completed 119 Budget unit cases, 29 PostgreSQL integration cases, seven app/authentication unit cases per clean build, one packaged WAR case, two existing planner journeys, four authentication phases and eight external review cases. The external class combines assurance and issue-characterization cases. The same app verifier also produced two byte-identical clean builds. [E3]

The recorded checks cover decimal bounds, zero versus unset, explicit saving, no-op history, local reload, account saves, historical labels and read-only history. Two authenticated sessions exercised stale month/catalog versions. Guest transfer checked preview/confirmation, conflict behavior and retention when clearing the local copy fails. These bounded controls are more informative than an undifferentiated green test count.

S3-01 / S2-01: confirmed save followed by failed refresh. After a real authenticated save returned saved=true, a browser bridge wrapper rejected the following month read. The editor displayed a read error, disabled Save, retained dirty state and returned failure to navigation despite the confirmed commit. The finding concerns acknowledgement and recovery semantics; it does not show that the database lost the save. A follow-up should retain confirmed revision metadata and distinguish confirmed save from refresh failure.

S3-02: rapid dialog reuse. After a dirty navigation prompt was cancelled, immediately reopening it allowed a queued close event from the first prompt to consume the new listener. The next Discard response then had no effective listener and did not navigate. A comparison that waited for the earlier dialog to close worked normally. The finding is a specific event-lifecycle race, not a claim that every dirty-navigation decision fails.

S3-03: interrupted history transfer. A synthetic source history contained income values 100, 200 and 300. When the second save failed before delegation, or returned Unknown before delegation, the stored prefix was 100; an explicit preview/confirmed retry produced account history 100, 300. When the second save committed and then returned Unknown, retry produced 100, 200, 300. No automatic write retry occurred.

Once a prefix exists, the implementation follows the existing-month merge path instead of restarting full replay. The final value can therefore arrive without reconstructing a missing middle account revision. Browser originals and the source payload remain preserved. This is a recovery limitation, not evidence of lost latest data or a broken promise of atomic multi-month transfer. Resumable full-history replay is a later product/contract decision.

7. PostgreSQL and custom ORM integrity

Step 4 completed six ORM unit tests, 75 ORM PostgreSQL tests, 112 SQL design checks and eight direct review probes. The 29 Budget integration tests had already passed in Step 3 and were reused without being counted again as new Step 4 executions. [E4]

Deferred reverse foreign keys enforce required Budget/line structure, guard/revision links, fixed lines and SET/amount companions. The completed checks cover real commit-time rejection, rollback of facts/guards/revisions, owner associations, versioned saves, consistent snapshots and connection cleanup. These support the tested invariants; they do not make every possible malformed state impossible.

S4-01: initial category facts. The explicit projection contract requires the earliest NAME, POSITION and STATUS facts to exist together at the same coordinate before deciding whether a category existed at a historical instant. The production Budget reader checks three as-of slots and skips the category if all are absent, without validating the initial coordinate.

Probes initialized and saved a normal catalog, then added one owned category through runtime INSERT rights without disabling constraints. A bare category and a future name/position pair without status were silently hidden. Three initial facts spaced one microsecond apart were accepted. Visible name/position without status correctly failed, and a valid later category behaved correctly in current versus pinned historical views. Normal saves write the initial facts atomically; these probes do not show ordinary saves creating corruption.

The ORM's passing malformed-category tests use a separate test projection helper. A targeted fix belongs in Budget's production reader with strict assertions against that reader, preserving the valid-later-category control. Passing a helper's tests cannot substitute for exercising its production consumer.

Real backend-termination probes also distinguished failure outcomes. A statement failure before commit was NOT_COMMITTED; terminating the backend before the commit call failed was UNKNOWN, even though an external read observed zero rows in that fixture. Each callback ran once, no write replay occurred, active leases returned to zero and subsequent pooled reads worked. A true wire-level lost acknowledgement after server commit was not tested; that path remains a driver-wrapper injection.

8. Security boundaries

Step 5 covered all 73 tracked app files at snapshot A, with baseline, focused and architecture/resource reviews. Budget and ORM consumers were traced for the relevant boundaries; this was not an exhaustive separate security audit of both module repositories. Multiple AI review passes are not independent external peer review. [E5]

Seven app unit cases, the packaged security/browser case, two planner journeys, four authentication phases and five additional security probes completed. Thirteen protected/alias paths remained closed under forged identity headers; public read/write controls behaved as expected. Tests rejected cross-session or stale CSRF tokens, foreign/null origins and malformed transfer requests. Forged owner/budget fields did not select another account, and a foreign-category save changed neither dataset's values or revisions.

Stored HTML/SQL-looking category text round-tripped literally, created no injected element and executed no script. Invalid Vaadin RPC tokens were rejected; logout denied replay of the old session. The source derives identity from the container principal, renews session/token state, uses secure cookie attributes and an Argon2id verifier file with restricted permissions. Missing account configuration failed closed in the local verifier.

No new reportable security vulnerability was found within those source and test boundaries. This is not certification of the deployed service, assurance against every attack or a current dependency-advisory inventory. The global login cooldown is a documented availability/guessing tradeoff; sustained session exhaustion was not measured. Plaintext browser-profile guest storage remains a disclosed boundary. Effective host grants, provider IAM, key custody and current public deployment settings were not attested.

9. Builds, tests and delivery

The accepted WAR/authentication outputs remained byte-identical across recorded clean builds. Step 6 additionally rebuilt ORM and Budget twice each; both pairs matched each other and the nested WAR JARs. The five accepted bundle digests and the two nested module JAR digests are retained in Appendix B. Correct JAR manifests identify source repositories and revisions. [E1, E6]

S6-01: publisher guards under Bash 3.2.57. The unchanged publisher relies on standalone conditional commands plus errexit to reject invalid endpoints, dirty source and missing/mismatched tags. In that native macOS shell, false standalone conditions did not stop the script. A narrow Maven wrapper redirected both synthetic endpoints to separate local file registries while executing the real deploy-file goal.

Missing tag, wrong tag, unapproved synthetic endpoint and a modified parent POM all returned exit zero and wrote local POMs. The modified POM differed from the tagged bytes while provenance still named unchanged HEAD/tag. The clean matching-tag control published the exact POM and sidecar. A separate Bash 5.2.37 control stopped a false condition; the full publisher was not run in that Linux shell. No hosted publication or unauthorized remote write was observed.

S6-02: Maven metadata. Effective library project/SCM URLs inherited app/orm and app/budget, with app.git/orm and app.git/budget connections and inherited tag v0.1.0. The JAR manifests remained correct. The defect affects source discovery and package/citation provenance rather than demonstrating wrong runtime classes.

S6-03: automatic gate. The app bootstrap executes library unit tests but skips the 75 ORM and 29 Budget PostgreSQL cases. The SQL harness and review-only probes are outside that gate; module repositories had no Actions workflows at inspection. Manual passing evidence therefore exceeds automatic regression coverage. No line/branch coverage percentage was measured or configured.

The frozen successful CI run printed accepted hashes but its artifact listing contained no retained binaries. Step 6 returned no tags or public releases for app, Budget or ORM, and did not verify hosted registry contents/authentication/immutability. The selected GitLab Maven registry was described as unprovisioned. These are dated observations, not claims that remote state cannot change.

10. Performance observations

Step 7 was one exploratory local run, without a supplied latency, throughput or recovery-time acceptance target. Three direct persistence fixtures used 10 categories/12 months/24 revisions per month, 50 categories/12 months/24 revisions, and 10 categories/one month/240 revisions. Each seed revision changed income and spending. Catalog facts were not repeatedly historized by that seeding. [E7]

Each direct operation had five warmups and 30 measured calls. Save preparation reads were outside the timed interval; writes added 35 revisions to the selected month. JDBC proxies counted prepared-statement executions while excluding SET LOCAL setup and driver transaction commands. Their overhead is part of direct timing. Unmodified server-pool browser measurements provide a separate application-path observation.

Direct operation	**10 categories / 24 revisions: p50 / p95 ms**	**50 categories / 24 revisions: p50 / p95 ms**
Catalog	15.78 / 17.93	73.53 / 89.08
Saved current month	28.70 / 33.19	126.76 / 134.47
Revision list	1.69 / 3.64	1.77 / 3.12
First historical revision	30.26 / 32.74	125.83 / 139.08
Atomic save	39.34 / 46.86	176.83 / 192.52

Catalog/current/save statement counts were 32/60/84 at 10 categories and 152/260/364 at 50; revision-list calls used one statement. For the tested fully saved active-category shape, catalog cost was $3C + 2$ statements and current projection $5C + 10$. Inactive categories or different persisted line shapes can alter counts. The 240-revision fixture's current read had p50/p95 29.29/36.21 ms and first revision 33.02/47.35 ms, still 60 statements. This finite fixture does not establish unbounded history or catalog-history scaling.

Browser operation	**10 categories: p50 / p95 ms**	**50 categories: p50 / p95 ms**
Warm saved-month RPC	37.41 / 44.42	136.25 / 144.17
Save RPC excluding follow-up read	38.68 / 42.76	136.82 / 148.36
Four-session burst, 60 reads	62.50 / 81.80	236.30 / 260.20

Warm operations explicitly selected January 2040. Save RPC timing excludes preparation/follow-up reads and client form rendering, so it is not click-to-editor-ready latency. Each four-session burst comprised four overlapping loops of 15 reads with no think time; both returned zero RPC errors. Their achieved rates were 62.24 and 16.83 reads/second over 964 and 3,566 ms, not sustainable or maximum capacity.

Five fresh-context guest page loads had p50/maximum 52.06/190.26 ms; login POST 31.02/64.33 ms; authenticated planner readiness 124.21/306.60 ms excluding login. Browser-process startup was excluded and the environment was warm. These page loads opened the then-current unsaved calendar month, while saved-month RPCs used seeded data.

All 707 raw latency samples and 397 half-second Jetty RSS samples are retained. Percentiles use nearest rank, including the field named median_ms. Peak sampled Jetty RSS was 350.17 MiB; that excludes the client harness, browser, database, Docker VM, proxy and OS and can miss short peaks. One 424,629-byte pg_dump took 182.78 ms with two overlapping reads; it did not include encryption, upload or restore.

S7-01 records category-dependent projection cost as an optimization candidate, not an SLO breach. S7-03 records the unverified combined constrained-Linux capacity boundary. Warm caches, sequential profiles, a single host, small samples and finite bursts prevent statistical tail, saturation, soak or production sizing claims.

11. Controlled failure and resilience findings

S7-02: executing-work deadlines. The app configures four pool connections and a 5,000 ms connection-acquisition timeout. The fixture reported PostgreSQL statement_timeout=0 and lock_timeout=0; direct JDBC network/query timeouts were zero. A held catalog lock produced a real PostgreSQL lock wait while an authenticated save remained pending beyond 6.5 seconds. After release it returned Saved at 6,683.80 ms and added exactly one revision. [E7]

Hikari's acquisition limit bounds waiting for a pool connection, not work already executing. PostgreSQL's zero statement/lock settings disable those limits. The finding is a medium resilience gap in the reviewed defaults; actual deployed overrides remain unverified. No agreed response target was demonstrated breached, and all four connections were not deliberately exhausted. Timeout/cancellation policy must preserve transaction cleanup and UNKNOWN outcome handling rather than add automatic write replay. [R2, R3]

Recovery probe	Observed result and timing boundary
Database stop/start	A protected read returned error=unknown; the anonymous guest route returned 200. The same container/volume became ready 195.04 ms after explicit start and the first explicit recovery read succeeded at 491.23 ms, preserving selected income and revision.
Owned JVM force-kill and explicit restart	The old session returned 401. Re-login and selected confirmed income/revision verification completed 2,103.19 ms after the harness began launching the same accepted app. One synthetic localStorage value remained byte-identical.

The database probe used container stop/start, not database crash, host/disk loss or a network blackhole. Recovery reads were explicitly issued by the harness. The JVM probe used an explicit harness restart, excluded failure-detection/kill-wait time and did not exercise systemd. Neither compares every financial value or proves unsaved editor recovery.

Three pilot harness defects were corrected before the completed Step 7 run: timestamp serialization, a changed automatically assigned Docker port and an unreaped child process. Pilot data are excluded. Those failures are not evidence of application data loss. Controlled restart success also does not resolve the shared-editor refresh defect.

12. Deployment and encrypted recovery

Step 8 made five anonymous read-only public requests at about 06:58 UTC on 8 November. HTTP root redirected 308 to canonical HTTPS; /login and /local returned 200; /transfer returned 401. HTTPS login still returned the login page without the spoof hostname when hostile Forwarded/X-Forwarded-Proto headers were supplied. No production sign-in, account read, restart, backup, restore or port scan was performed. [E8]

These point-in-time results establish the observed public boundary, not deployed WAR identity, actual loopback listeners/firewall rules, certificate renewal or uptime. The checked-in Caddy, PostgreSQL, role and systemd templates were reviewed, not asserted to be current installed configuration.

The checked-in isolated recovery drill passed, covering encrypted weekly/pre-change backups, ciphertext readback, local OK/FAILED mail, calendar/lifecycle configuration, owner/grant restoration and rejection of wrong age keys, damaged objects and existing targets. Because its history schema is a demonstration, the external Step 8 fixture added Budget V1 SQL and runtime grants extracted from the accepted WAR.

The additional fixture seeded all 24 real tables with 47 synthetic rows, two revisions, a category rename, unset amounts and explicit zero/value facts. A real pg_dump/age/AWS CLI backup was uploaded/read back in Moto, fetched/decrypted and restored to a new PostgreSQL 18.6 cluster. Every table's sorted-row SHA-256/count and the schema/relation owner/ACL fingerprint matched. Runtime-role SQL projections preserved the selected current and historical values.

The backup user could not read the private age identity. A closed local mail relay made report-backup exit one despite a verified backup receipt. Import without finance_owner failed and left a newly created database without the Budget schema; retry against that existing empty target was refused. Applying roles and retrying in a fresh target succeeded. This is the documented createdb versus single-transaction import boundary, not an unexpected partial-data restore. [R4]

The combined drill container took 18.936 seconds, excluding image building. The local fetch/new-cluster/negative-case/retry/SQL-validation phase took 2.345 seconds. These are not operational RTOs. They omit provisioning, real provider transfer, full application startup/verification, human work, DNS/proxy cutover and acceptance. The added phase applied SQL directly; it did not run Flyway's ledger, SCRAM login or a recovered browser journey.

S8-01 retains the deployed provenance/control gap, including S1-01. S8-02 retains absent fresh evidence for the actual weekly job, independently recoverable off-VM key/bundle and verified receipt, live provider expiration, real inbox delivery, complete timed recovery, billing period and human release acceptance. It does not assert that private operational work or records do not exist. Issue #6 was open when inspected in Step 8.

The runbook's approximate seven-day-plus-completion loss window depends on successful weekly backups; missed jobs enlarge it. Its four-hour recovery target starts when the operator begins recovery and extends through verified access/cutover. Neither target is established by the tiny local drill. Source rollback procedures distinguish keeping the database with a compatible previous app from restoring a separate cluster for incompatible schema. A rollback that would discard subsequent committed writes requires an explicit decision about the acceptable data-loss window.

13. Consolidated findings and disposition

Original IDs remain usable across the reports. Source/runtime defects, limitations, cost observations and missing evidence are not combined into one severity score. All findings below remain unresolved for the frozen runtime; preparing report metadata only addresses this document's own publication preparation.

IDs / class	Consolidated finding	Disposition for a later release
S1-01, S8-01 / evidence gap	Deployed source, bytes and effective controls are unidentified by this review.	Compare a protected deployed manifest/configuration record with the accepted candidate; publish a sanitized summary.
S1-02 / evidence gap	Successful CI exposes hashes without retained candidate binaries.	Retain exact retrievable artifacts, manifest and runtime references.
S1-03 / delivery evidence	Declared coordinates do not establish tags, releases or registry packages.	Verify approved remote project/version state and retain readback evidence before publication.
S1-04 / metadata gap	Source snapshots lack explicit code licensing/citation files.	Select software licensing and attribution before publication. This report's CC BY 4.0 and author metadata do not relicense code.
S2-01 = S3-01 / medium defect	Refresh failure obscures an already confirmed save.	Preserve revision acknowledgement and test post-commit read failure against the shared planner.
S2-02 / low defect	App's direct ORM/Hikari usage is declared only transitively.	Align direct build dependencies with composition ownership.
S3-02 / medium defect	Rapid unsaved-dialog reuse can ignore Discard.	Correct event/listener lifecycle and keep the ordinary-navigation control.
S3-03 / recovery limitation	Retrying partial history replay can omit a middle account revision while retaining latest value/originals.	Document latest-only retry; choose whether resumable history is required.
S4-01 / medium defect	Production catalog projection omits required initial-fact validation.	Validate the three malformed fixtures in Budget's real reader, preserving valid history.
S6-01 / medium defect	Bash 3.2 publisher guards allow invalid fixtures to reach local deployment.	Use explicit rejection/supported shell and verify all four negative guard cases.
S6-02 / low defect	Effective module POM source/SCM metadata points under app.	Correct module metadata and compare published POM, JAR and provenance.
S6-03, B3 / validation gap	Persistence integration and review probes are outside the automatic gate.	Gate relevant suites and turn accepted fixes' characterization cases into strict regression checks.
S7-01 / measured cost	Saved-month prepared statements grow with categories.	Evaluate bulk projection within Budget while retaining owner/snapshot/as-of semantics.
S7-02 / medium resilience gap	Pool acquisition timeout does not bound executing lock/query work.	Select/test execution and cancellation policy without automatic unknown-outcome writes.
S7-03 / validation gap	Combined constrained-Linux CPU/ memory/disk/backup headroom is unverified.	Measure the accepted workload on comparable approved resources.

IDs / class	Consolidated finding	Disposition for a later release
S8-02, B1 / acceptance gap	Current provider scheduling, retention, recovery and human acceptance are unverified.	Inspect fresh operational evidence and time complete off-VM recovery through app access.
B2 / product proposal	Browser-local guest data lacks a portable backup path.	Decide intended public use and evaluate export/restore as a separate product decision.
B4 / usability proposal	Single-account sign-in expectations are unclear.	Clarify visitor-facing access/storage wording and verify comprehension if consequential.

S2-01 and S3-01 describe one confirmed-save problem at different evidence stages; they must not be counted as separate defects. Business concerns B1/B3 overlap operational/CI gaps. No vulnerability is added to this register solely because a correctness or availability finding exists.

14. Release implications and proposed order

The proposed next release focuses on the reproduced UI, projection and delivery defects, together with the missing acceptance evidence. This sequence reflects the observed cases rather than a new delivery commitment.

Corrective work requires regression assertions for the intended behavior, alongside the existing stale-version, account-isolation, pinned-history and unknown-outcome controls. The acceptability of limited history transfer and browser-local guest storage depends on the intended audience. Merging a finding's report does not resolve the underlying finding.

Release verification should align automatic checks with persistence and workflow claims, publisher rejection cases, effective Maven provenance and retained candidate bytes. No line-coverage target or architecture rewrite follows from these findings.

The existing release-preparation plan orders dependencies as build parent, ORM, Budget and app/authentication/WAR. Reviewed coordinates are parent 0.1.0, ORM 0.1.0, Budget 0.2.0 and app/authentication 0.1.0. Proposed tags and versions still require approved release commits and checks that the registry coordinates are unused. Fixes may require changed versions/pins. The existing GitLab Maven decision is retained; no alternate registry is silently selected. [E9]

Operational acceptance requires deployed-artifact correspondence, workload and backup capacity measurements, verified off-VM recovery material and a completed recovery exercise. Publication of the report is separate from software package releases and production cutover.

15. Assistance, authorship and accountability

Anton Serdyuchenko defined the review scope and publication objectives and is responsible for the report's authorship and editorial decisions. OpenAI ChatGPT/Codex substantially assisted source inspection, test design and execution, fault injection, analysis, drafting and LaTeX preparation.

Activity	Author contribution	AI assistance / execution
Scope and progression	Defined the review scope and publication objectives.	Developed the review sequence and performed the recorded technical analysis.
Technical evidence	Provided deployment-context information, including the unavailable manifest.	Developed and executed probes, diagnosed fixture failures and documented evidence limits.
Review records	Review progression and merges are recorded in the project history.	Prepared review documents and verified GitHub merge status.
Consolidation	Report authorship and editorial review.	Reconciled findings, prepared document sources and checked compilation and layout.

Contribution percentages, causal productivity effects and individual performance were not measured. Multiple AI review passes do not constitute external expert review or independent reproduction. ChatGPT/Codex are assistance systems, not authors.

Final factual review, licensing and attribution, and the publication decision remain the author's responsibility. Production release acceptance and independent reproduction are not established by this publication candidate.

16. Limitations

Study and product scope. One project and a bounded set of synthetic workflows were reviewed. There is no representative user study, commercial experiment, control group or blinded independent assessment. Architecture adequacy is assessed against the selected design, not proved superior to alternatives.

Runtime correspondence. Exact reviewed source and local artifact bytes are known. The running public service's commit, bytes, effective roles, configuration and infrastructure remain unmatched. A source finding must not be asserted to describe today's production bytes without that comparison.

Coverage and failure models. Passing selected tests does not enumerate every race, corrupted state, browser/platform or attack. Artificial bridge/driver/port decorators and synthetic malformed rows are disclosed. Backend termination, container stop/start and explicit JVM restart exercise different failure models from real post-commit packet loss, power loss or complete host recovery.

Measurement. No predefined service-level objectives or statistical protocol were supplied. The exploratory sample count, warm cache/JIT state, single host, finite bursts and partial memory accounting do not establish stable p95, maximum throughput, leak freedom or the planned VM's suitability.

Operations. Moto lifecycle readback is not elapsed provider expiration. Local SMTP is not actual authenticated/TLS mailbox delivery. Calendar parsing is not a running timer, and ordinary container processes are not systemd PID 1 or an installed sandbox. A private key protected within a disposable container is not proof of off-VM independent custody.

Reproduction and drift. No independent person/host reproduced the entire review. Maven/ browser caches and public dependency/package repositories were reused. Image IDs and artifact hashes improve traceability but do not guarantee future network availability or full hermetic reconstruction.

Records and assistance. Total review effort, AI usage/cost and real operating cost were not established for this study. Repeated suite runs are not independent trials. The initial business record is consolidated from the recorded assisted review; its live journeys were not repeated for this manuscript. Private operator records were not audited in the eight-step public review.

17. Reproducibility and future validation

The canonical detailed evidence is snapshot D's docs/review directory. Each step links its external runner, case source, sanitized results and limits. Step 3 supplies workflow cases; Step 4 direct integrity/connection probes; Step 5 security probes and coverage record; Step 6 reproducibility/ publisher cases; Step 7 measurements/failures; Step 8 encrypted recovery with the actual schema. [E3-E8]

Use disposable synthetic environments and the pinned runtime sources, not a currently deployed database. Preserve the accepted binary digests and module pins when comparing results. Run each step according to its own prerequisites: full source checkouts, appropriate Java/Jetty/PostgreSQL tools, isolated caches and Docker/browser fixtures. Some steps reuse completed evidence instead of rerunning prior suites.

A passing characterization should be expected to change after a fix. Regression tests for accepted fixes must assert the intended corrected behavior rather than merely duplicate the old failure. Preserve negative controls and transaction semantics when reducing query count or introducing deadlines.

For a later publication version, report the actual fix revisions, updated dependencies, changed acceptance decisions and which cases were rerun. Retain individual performance runs and measurement boundaries before drawing comparative conclusions. New production-readiness statements require current protected manifest/configuration, provider and recovery evidence with an approved sanitized public summary.

18. Evidence table and revision anchors

A, B, O and D denote the full commits in Section 3. The report links to D for the retained review record, while A/B/O identify the software examined. These anchors are not claims that every historical command ran at D or that D was deployed.

Evidence	Record and supported scope
E0	business-review.md in this publication bundle: sanitized consolidation of the initial 7 November assisted guest/product review, including synthetic values, observations, recommendations and limits.
E1	01-baseline.md: source/build pins, CI digests and deployment correspondence.
E2	02-architecture.md: production inventory, ownership, transactions and S2 findings.
E3	03-workflows.md, results and reproduction: assurance and failure-characterization cases.
E4	04-postgresql-orm.md, results and reproduction: SQL/ORM suites, catalog defect and backend termination.
E5	05-security.md, results and coverage: scoped source audit and boundary probes.
E6	06-builds-tests-delivery.md, results and reproduction: artifacts, publisher guards, metadata and automatic gate.
E7	07-performance-resilience.md, raw results and reproduction: profiles, query counts, RSS and controlled failures.
E8	08-deployment-recovery.md, results and reproduction: public requests, operating controls and encrypted schema recovery.
E9	release-preparation.md: proposed dependency/version mapping and acceptance/ publication prerequisites.
E10	Candidate CI run, PR #12, issue #6: dated CI and mutable prior implementation/acceptance context, as inspected in the step reports.

The supplied evidence-manifest.json records report/dataset paths, D-pinned URLs and SHA-256 values. It supports checking retrieval without replacing the source and reproduction instructions. Evidence E0 is retained with this publication because the initial review preceded the step-by-step GitHub reports.

References

- **R1.** Serdyuchenko, A. (2026). Foundations for AgentOps: Linux Diagnostics and a Read-Only YandexGPT Tool Call (Version 1.0) [Technical report]. Zenodo.
- **R2.** HikariCP. Configuration: connectionTimeout. Pool-acquisition semantics; consulted during Step 7, 8 November 2026.
- **R3.** PostgreSQL 18. Client connection defaults: statement_timeout and lock_timeout. Timeout semantics; consulted during Step 7, 8 November 2026.
- **R4.** PostgreSQL 18. pg_restore. Single-transaction import boundary; consulted during Step 8, 8 November 2026.
- **R5.** Zenodo. Create new upload and Resource types. Deposit preparation guidance; accessed 8 November 2026.
- **R6.** Creative Commons. Attribution 4.0 International. License for this report and document sources; accessed 8 November 2026.

Appendix A. Proposed Zenodo deposit metadata

Title: Finance Lab MVP Review: Product Findings, Technical Integrity and Release Evidence

Author / creator: Serdyuchenko, Anton. ORCID 0000-0001-7770-2201. No affiliation is asserted. AI assistance is disclosed in Section 15.

Resource type: Publication / Report (Technical Report). Pilot Engineering Study is a descriptive classification, not a claim of peer review.

Version and language: 1.0; English.

Dates and status: Review observations 7-8 November 2026; report preparation 8 November 2026. The actual public release date must be supplied when publishing. This is a publication candidate; no Zenodo draft, reserved DOI or published record is asserted.

License: CC BY 4.0 for this report, its Markdown/LaTeX sources and document metadata. The app, Budget and ORM snapshots' code licensing decisions remain separate and unresolved in the review.

Description: Use the Abstract, followed by the technical-report/pilot-study classification, synthetic-data statement and AI-assistance disclosure. The metadata draft omits publication date and DOI instead of inventing them.

Related identifiers: Documents the A/B/O software snapshots and references the D review evidence and the related engineering report R1. No version relationship is assigned between the two reports.

Deposit files: PDF, standalone main.tex and equivalent report.md; retain business-review.md, evidence-manifest.json and metadata-draft.json with the source bundle. SHA256SUMS identifies the prepared files. Final DOI/date changes require rebuilding the PDF and updating the checksums before deposit.

A Zenodo deposit would archive the report; it would not perform software release acceptance or provide peer-review validation.

Appendix B. Accepted artifact digests

These digests identify local/CI artifacts compared in the review, not bytes independently read from the production server.

Artifact	SHA-256
application/target/ finance-lab.war	`a2a81f71a567587934cf7c830b3adf2b4c10d4fa1e9004f9fc60fbfdba27307a`
authentication/target/ authentication-0.1.0.jar	`661f6f65ccbde13e31cc432870ce16bb8e0e466cede7b8235bce8dd505645a24`
authentication/target/ lib/password4j-1.8.4.jar	`d1406a158533b65bda7c20cebf7bf2a9a0626f8931fd1dc3d39243af5849edf5`
config/jetty-auth.xml	`fd8b7ab5d9393f7e1e945e6df0f384c4ac96a8da878d3dbc0ffaf28773cfb494`
build-parent/pom.xml	`aa61d398a42e3df231b696532a545872296bba8d039b453cf5ad98f5b284a34f`
Nested orm-0.1.0.jar	`f172af011a04d74549cb917f2bab8feb4bd79ad1594fdee8b71815db2f1145e4`
Nested budget-0.2.0.jar	`b24dfb9fb4d2a9a8ff1f2edcef7984b1b14f1e0542654c37f251752bb62fb730`